

Tarefa 5: Desafios Iniciais da Programação Paralela

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Junho de 2026

1. Introdução

A programação paralela permite reduzir o tempo de execução de algoritmos computacionalmente intensivos ao distribuir o trabalho entre múltiplos núcleos de processamento. Entretanto, a simples inserção de diretivas de paralelismo **não garante nem correção nem eficiência**. Esta tarefa ilustra, de forma prática, os dois desafios clássicos enfrentados por programadores ao paralelizar código com OpenMP: (i) a **condição de corrida**, que compromete a correção do resultado, e (ii) o **desbalanceamento de carga**, que limita o ganho de desempenho.

O problema escolhido é a contagem de números primos no intervalo $[2, N]$, com $N = 10.000.000$. A função de teste de primalidade `is_prime(n)` possui custo $O(\sqrt{n})$, o que torna o problema propositalmente desequilibrado quando particionado em blocos contíguos — propriedade que evidencia com clareza ambos os desafios.

2. Ambiente Experimental

Item	Valor
Processador	AMD Ryzen 5 5600G
Núcleos físicos	4 (8 threads lógicas com SMT)
Compilador	GCC com -O2 -fopenmp
N (tamanho)	10.000.000
Sistema de temporização	clock_gettime(CLOCK_MONOTONIC)

3. Implementação

3.1 Versão Sequencial

O código sequencial percorre o intervalo e acumula o contador de forma segura, sem qualquer risco de acesso concorrente:

```
long count = 0;
for (int i = 2; i <= N; i++)
    if (is_prime(i))
        count++;
```

3.2 Versão Paralela Ingênua

A versão paralela foi obtida inserindo exclusivamente a diretiva `#pragma omp parallel for` antes do laço. A lógica interna permanece inalterada, expondo intencionalmente os dois problemas:

```
long count = 0;
#pragma omp parallel for          /* única modificação */
for (int i = 2; i <= N; i++)
    if (is_prime(i))
        count++;                /* RACE CONDITION */
```

A instrução `count++` não é atômica: ela se expande em três operações de máquina distintas (`LOAD` → `ADD` → `STORE`). Múltiplas threads podem executar esse trecho simultaneamente, levando a **atualizações perdidas (lost updates)**.

4. Resultados

Os três blocos a seguir mostram execuções consecutivas do mesmo binário. O valor de referência correto é **664.579 primos**.

Execução 1

Versão	Primos	Tempo (s)	Correto?
Sequencial	664.579	1,2323	Sim
Paralela ingênua – 2 thr	661.667	0,7741	NÃO
Paralela ingênua – 4 thr	661.978	0,4686	NÃO
Paralela ingênua – 8 thr	662.020	0,4364	NÃO

Execução 2

Versão	Primos	Tempo (s)	Correto?
Sequencial	664.579	1,2348	Sim
Paralela ingênua – 2 thr	661.629	0,7945	NÃO
Paralela ingênua – 4 thr	661.655	0,4504	NÃO
Paralela ingênua – 8 thr	662.471	0,3546	NÃO

Execução 3

Versão	Primos	Tempo (s)	Correto?
Sequencial	664.579	1,2390	Sim
Paralela ingênua – 2 thr	661.809	0,8369	NÃO
Paralela ingênua – 4 thr	661.956	0,4463	NÃO
Paralela ingênua – 8 thr	661.851	0,4351	NÃO

5. Análise dos Desafios

5.1 Desafio 1 — Condição de Corrida

A instrução `count++` é uma operação read-modify-write não atômica. Ao paralelizar o laço, múltiplas threads acessam e modificam a variável `count` sem sincronização, conforme a sequência abaixo:

Thread A	Thread B
LOAD <code>count</code> → <code>regA</code>	LOAD <code>count</code> → <code>regB</code>
ADD <code>regA</code> , 1	ADD <code>regB</code> , 1
STORE <code>regA</code> → <code>count</code>	STORE <code>regB</code> → <code>count</code> ← <i>overwrites A</i>

O resultado é que **incrementos são silenciosamente perdidos**, produzindo um contador final sempre inferior ao valor correto (664.579). Os valores obtidos — como 661.629 ou 662.471 — variam a cada execução porque a interleaving de instruções de threads é não-determinística. Esse comportamento é o sintoma clássico de uma **condição de corrida**.

A correção exige proteger o acesso compartilhado, por exemplo com `reduction(+:count)`, `#pragma omp atomic` ou variáveis locais somadas ao final.

5.2 Desafio 2 — Desbalanceamento de Carga

O escalonamento padrão `schedule(static)` divide o intervalo $[2, N]$ em blocos contíguos de tamanho aproximadamente igual. Porém, verificar a primalidade de n custa $O(\sqrt{n})$ — números maiores exigem mais iterações. As threads que recebem os blocos finais do intervalo trabalham significativamente mais do que aquelas que recebem os blocos iniciais.

Isso explica o **estagnação do speedup** ao passar de 4 para 8 threads: os tempos de execução com 4 e 8 threads são praticamente iguais ($\approx 0,44$ s). As threads que processam números menores concluem rapidamente e ficam ociosas enquanto as que processam os blocos finais ainda trabalham — o gargalo recai sobre as últimas threads.

A solução é utilizar `schedule(dynamic)` ou `schedule(guided)`, que atribuem iterações dinamicamente e permitem que threads mais rápidas assumam trabalho das mais lentas.

6. Síntese

Desafio	Causa	Sintoma
Condição de corrida	<code>count++</code> não é atômico (load-modify-store)	Resultado errado e não-determinístico
Desbalanceamento de carga	Custo de <code>is_prime(n)</code> cresce com $\sqrt{n}$; <code>schedule(static)</code> divide blocos contíguos	Speedup estagna ao ultrapassar 4 threads

Os resultados demonstram que paralelizar código com **uma única linha** de diretiva OpenMP — sem atenção à semântica de acesso compartilhado e à distribuição de carga — produz um programa que é ao mesmo tempo **incorreto e ineficiente**. A programação paralela correta exige raciocínio explícito sobre esses dois aspectos antes de qualquer medição de desempenho.

7. Conclusão

Esta tarefa evidenciou, de modo prático e reproduzível, que a paralelização de um laço com OpenMP introduz dois desafios fundamentais que devem ser tratados de forma independente. O primeiro é a **correção**: qualquer operação compartilhada que não seja atômica pode produzir resultados errados de forma silenciosa e não-determinística. O segundo é a **eficiência**: uma distribuição estática de iterações ignora a heterogeneidade do custo de cada iteração, desperdiçando capacidade de processamento.

A versão paralela ingênua, apesar de reduzir o tempo de execução em relação à versão sequencial (**speedup de $\approx 2,8\times$** com 4 threads), retornou resultados incorretos em todas as execuções e não escalou além de 4 threads. Esses dois sintomas, tomados em conjunto, ilustram com precisão os riscos de paralelizar código sem o devido cuidado com sincronização e balanceamento de carga.

8. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <math.h>
4  #include <time.h>
5  #include <omp.h>
6
7  #define N 1000000
8  #define BILLION 1000000000.0
9
10 static double elapsed(struct timespec a, struct timespec b) {
11     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
12 }
13
14 static int is_prime(int n) {
15     if (n < 2) return 0;
16     if (n == 2) return 1;
17     if (n % 2 == 0) return 0;
18     int lim = (int)sqrt((double)n);
19     for (int d = 3; d <= lim; d += 2)
20         if (n % d == 0) return 0;
21     return 1;
22 }
23
24 /* — Versão sequencial — */
25 static long contar_sequencial(void) {
26     long count = 0;
27     for (int i = 2; i <= N; i++)
28         if (is_prime(i))
29             count++;
30     return count;
31 }
32
33 /* — Versão paralela INGÊNUA — lógica original inalterada — */
34 Problema 1: condição de corrida em count++ (read-modify-write não atômico)
35 Problema 2: schedule(static) distribui blocos contíguos, mas verificar
36 primalidade de números grandes custa mais → últimas threads
37 recebem carga desproporcional.
38 static long contar_paralelo_ingnuo(int n_threads) {
39     long count = 0;
40     omp_set_num_threads(n_threads);
41     #pragma omp parallel for
42     for (int i = 2; i <= N; i++)
43         if (is_prime(i))
44             count++; /* RACE CONDITION */
45     return count;
46 }
47
48 int main(void) {
49     struct timespec ini, fim;
50
51     /* sequencial */
52     clock_gettime(CLOCK_MONOTONIC, &ini);
53     long seq = contar_sequencial();
54     clock_gettime(CLOCK_MONOTONIC, &fim);
55     double t_seq = elapsed(ini, fim);
56
57     printf("Contagem de primos entre 2 e %d\n", N);
58     printf("%-25s %-10s %-10s %-8s\n",
59           "Versão", "Primos", "Tempo(s)", "Correto?");
60     printf("%-25s %-10s %-10s %-8s\n",
61           "-----", "-----", "-----", "-----");
62     printf("%-25s %-10ld %-10.4f %-8s\n",
63           "Sequencial", seq, t_seq, "sim");
64
65     int threads[] = {2, 4, 8};
66     for (int k = 0; k < 3; k++) {
67         int t = threads[k];
68         char label[32];
69         snprintf(label, sizeof(label), "Paralela ingênua (%d thr)", t);
70
71         clock_gettime(CLOCK_MONOTONIC, &ini);
72         long par = contar_paralelo_ingnuo(t);
73         clock_gettime(CLOCK_MONOTONIC, &fim);
74         double t_par = elapsed(ini, fim);
75
76         printf("%-25s %-10ld %-10.4f %-8s\n",
77               label, par, t_par, par == seq ? "sim" : "NAO");
78     }
79
80     return 0;
81 }
```